Café de la Plaza

Aplicación web y pruebas de extremo a extremo con Cypress
Tarea universitaria - 23 de octubre de 2026

Repositorio público
https://github.com/jefroMMM/E2E-Tarea

Ejecución exitosa de GitHub Actions verificada
https://github.com/jefroMMM/E2E-Tarea/actions/runs/40037388136

Resultado real

Cypress local: 3 de 3 pruebas aprobadas, 0 fallos. Orden inverso 03, 02, 01: 3 de 3 aprobadas en procesos separados. TypeScript sin errores. GitHub Actions: ejecución real exitosa, incluida la suite completa, el orden inverso y el artifact de capturas y videos.

Aplicación: HTML, CSS y JavaScript. API: Node.js y Express. Persistencia: SQLite en archivo, con transacciones para guardar el pedido y descontar existencias. Los precios se manejan en centavos.

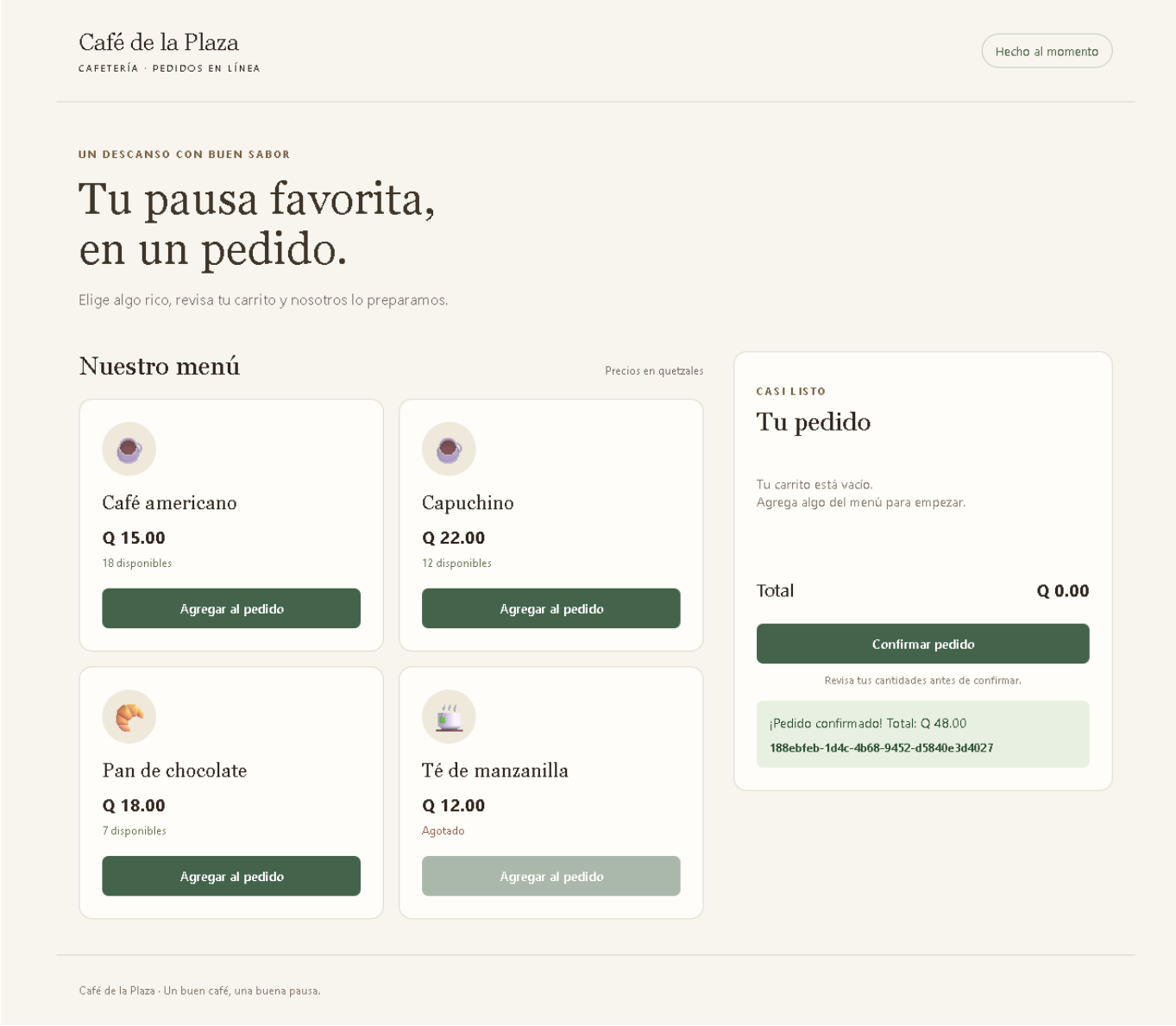
Video de demostración (máximo 3 minutos)

PENDIENTE: el estudiante debe grabar y publicar el video. Seguir guion_video.md; verificar que el docente pueda abrir el enlace. Después, completar video_url en estado.json y regenerar este PDF con generar_pdf.py. No se ha inventado un enlace.

ChatGPT y Codex fueron utilizados para apoyar el desarrollo, proponer pruebas, revisar selectores y documentación. Las pruebas tienen pasos y aserciones explícitos para facilitar su explicación.

1. Pedido exitoso

Se reinicia la base, se agregan café y pan, y se modifica el café a 2 unidades. El total esperado es Q 48.00. Cypress inspecciona el POST, verifica HTTP 201 y el UUID visible; consulta el GET del pedido para comprobar total y líneas guardadas, y comprueba el stock actualizado.

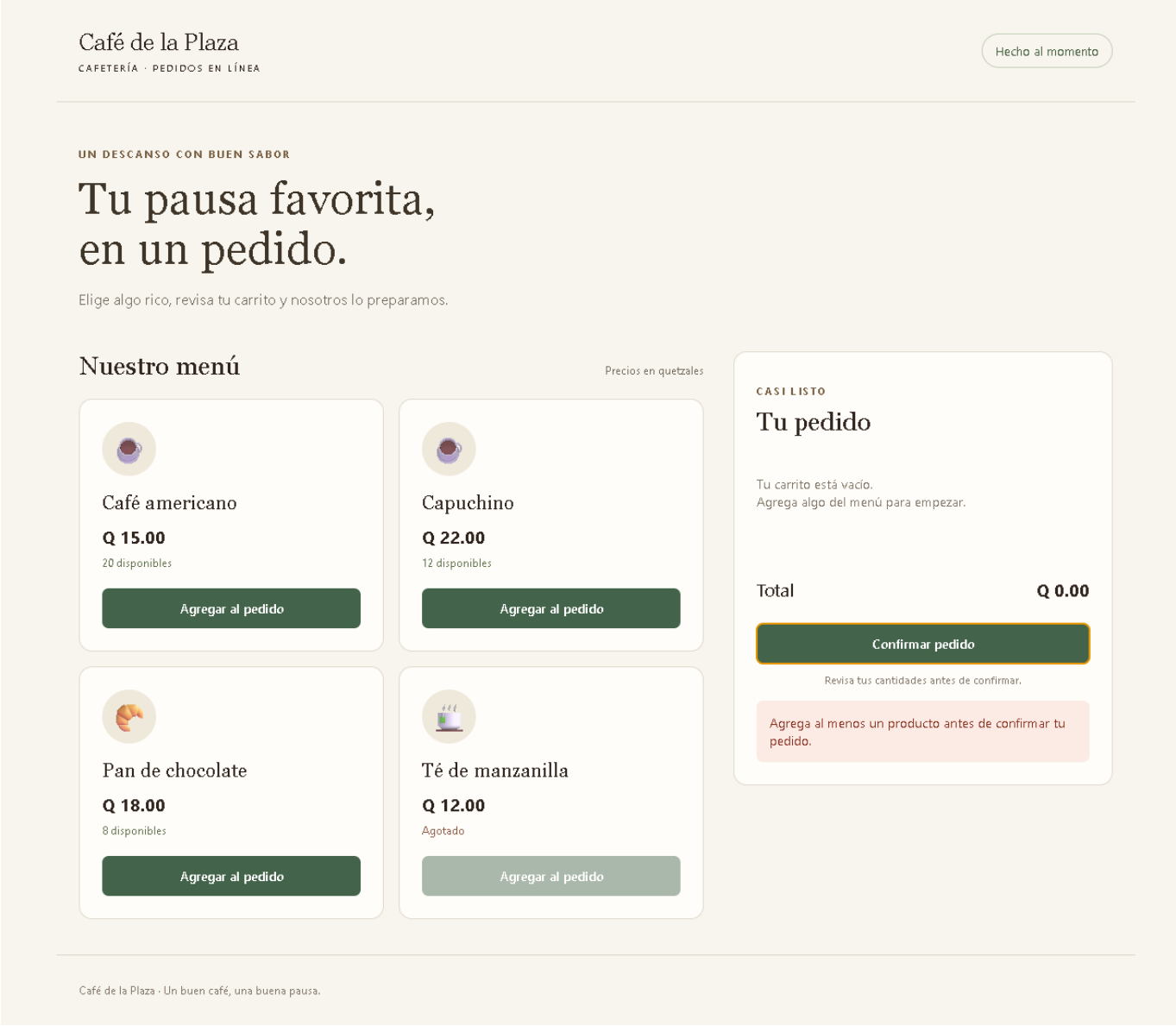


Captura real tomada por Cypress tras la confirmación. El identificador es único para esa ejecución; cambia al repetir la prueba.

Evidencia adicional: 01-carrito-exitoso.png muestra las cantidades antes de confirmar.

2. Validación del carrito vacío

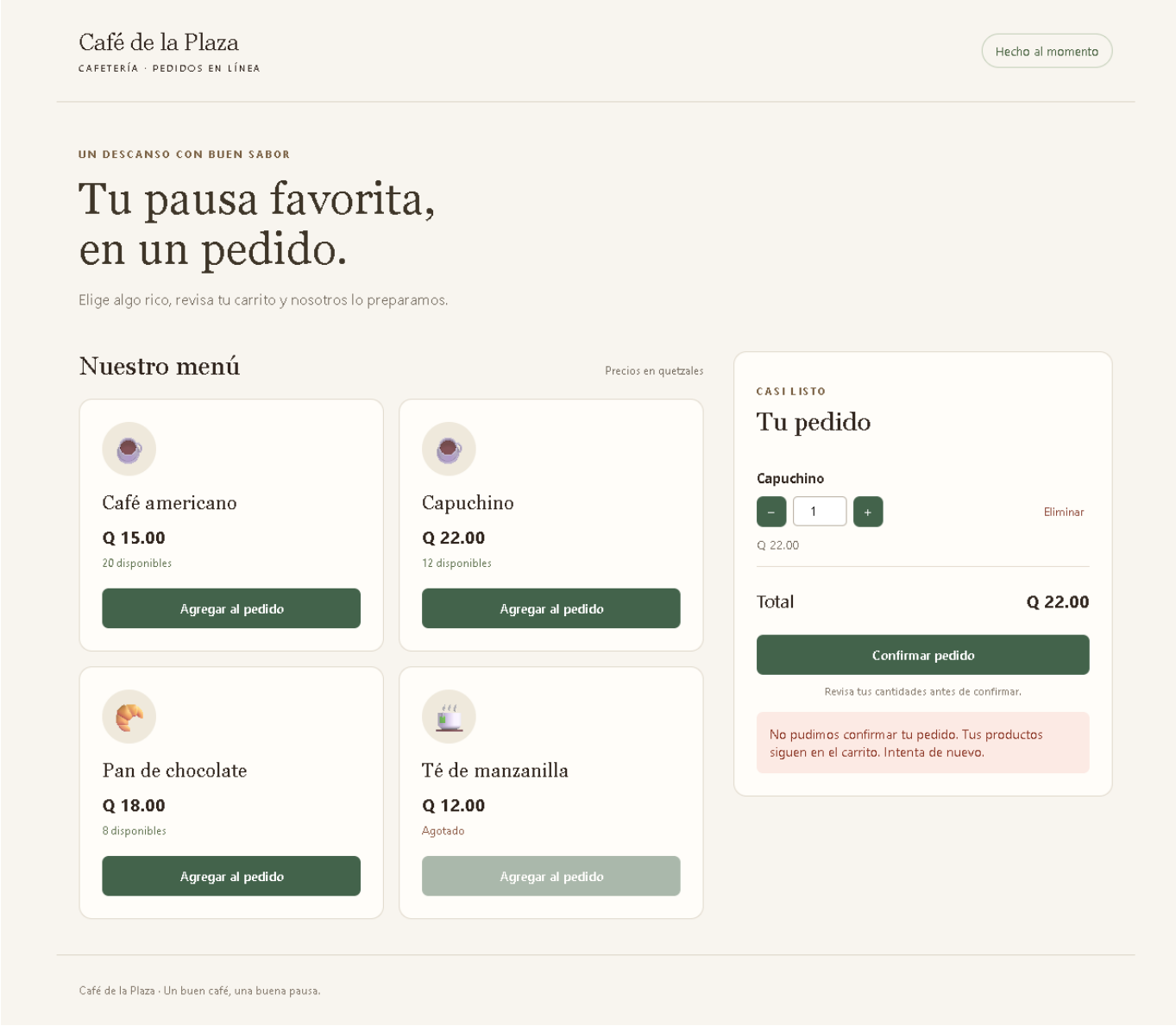
Tras reiniciar los datos, se intenta confirmar sin productos. La interfaz muestra la validación. Cypress comprueba que el historial de POST interceptados está vacío y que GET /api/orders devuelve una lista vacía.



Captura real del mensaje de validación. No se creó ningún pedido.

3. Fallo controlado y recuperación

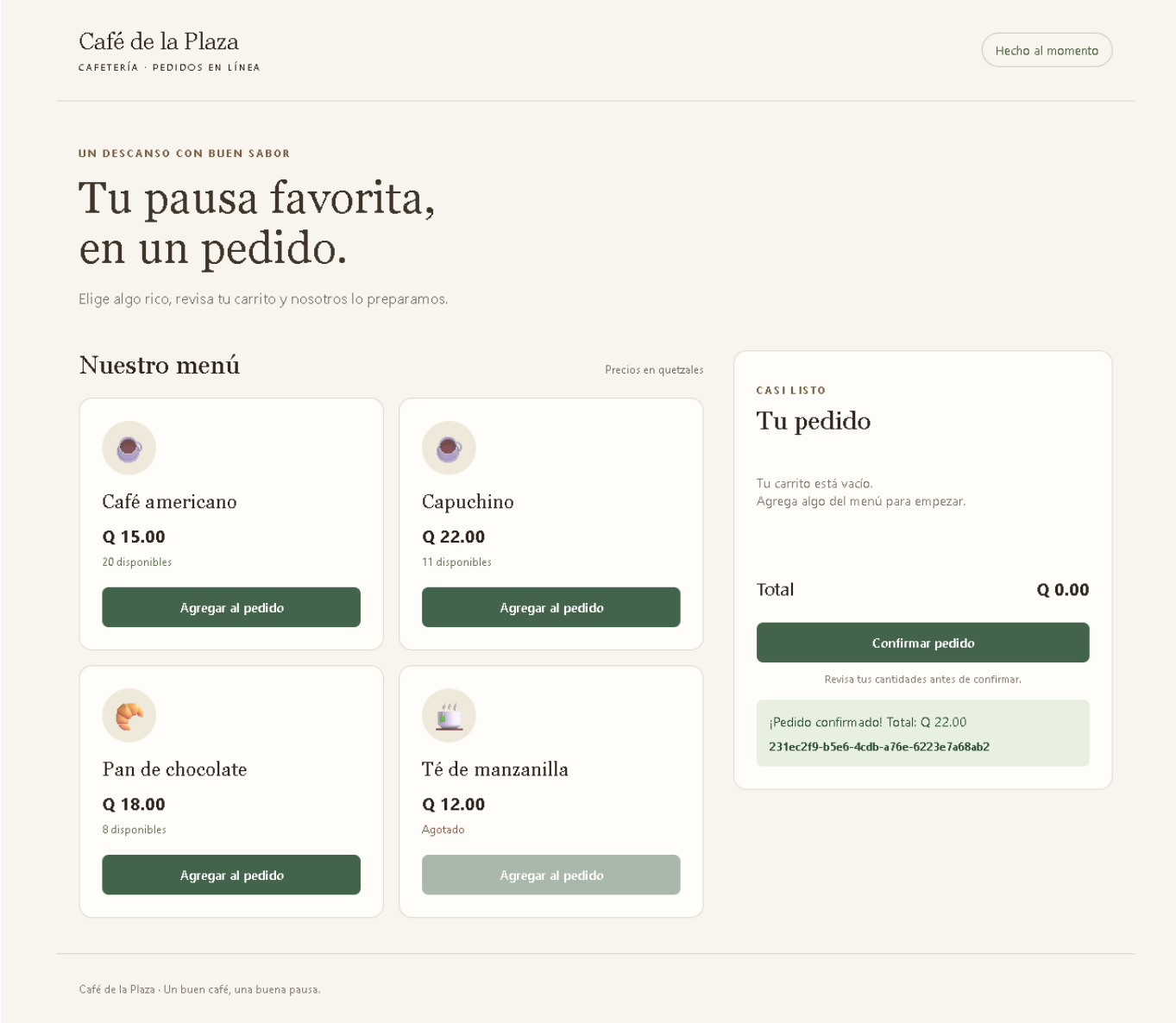
Se agrega un capuchino y cy.intercept configura una sola respuesta HTTP 500 con times: 1. Cypress comprueba método, URL, cuerpo y código HTTP. El error se explica y el carrito conserva el producto y Q 22.00.



El fallo de esta solicitud es simulado. El servidor y la base de datos siguen disponibles; todavía no hay pedidos guardados.

3. Recuperación con la API real

En el segundo intento deja de aplicarse la simulación. La API real responde 201 y guarda el pedido en SQLite. Cypress verifica el identificador, el total y que existe exactamente un pedido.



Captura real del reintento completado. El carrito se vacía solamente después de recibir la confirmación.

Verificación, datos y estabilidad

Cypress local: 3 de 3 pruebas aprobadas, 0 fallos. Orden inverso 03, 02, 01: 3 de 3 aprobadas en procesos separados. TypeScript sin errores. GitHub Actions: ejecución real exitosa, incluida la suite completa, el orden inverso y el artifact de capturas y videos.

Los archivos resultados-cypress.txt y resultados-inverso.txt contienen la salida real de las ejecuciones. verificacion-manual.md y verificacion-api.txt documentan las verificaciones adicionales. Se confirmó también la persistencia después de reiniciar el proceso del servidor.

Reinicio e independencia

Cada beforeEach llama POST /api/test/reset. Esta ruta existe solo con NODE_ENV=test y restaura cuatro productos y cero pedidos en data/test.sqlite. La aplicación normal usa otra base. npm run seed y npm run seed:test permiten restaurar cada base desde terminal; ambos borran los pedidos de su base.

Prevención de flakiness

Se usan selectores data-cy, testIsolation, alias de red registrados antes de las acciones y aserciones reintentables. No hay cy.wait con pausas numéricas. Cada recorrido prepara sus datos y también se ejecutó por separado en orden inverso. Los botones se bloquean durante la confirmación.

Qué es real y qué se simula

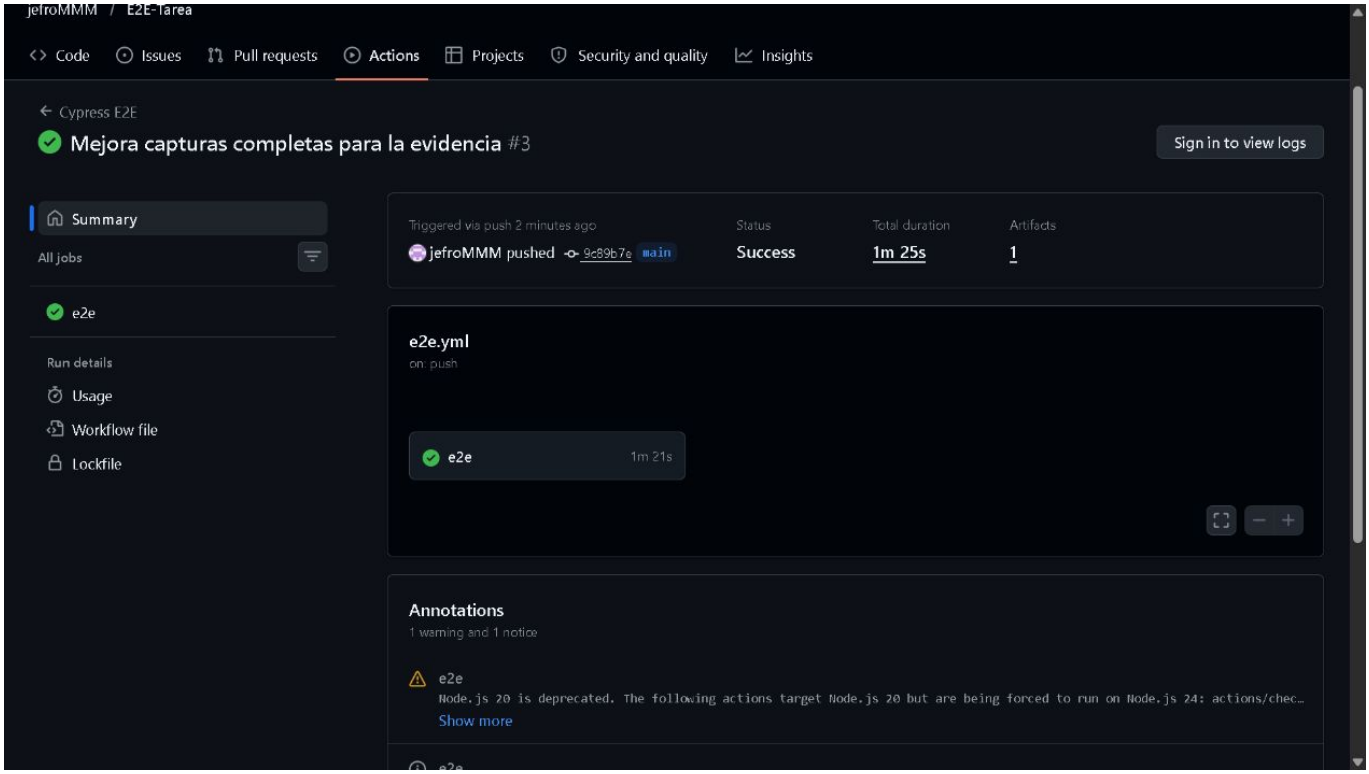
El frontend, Express, HTTP y SQLite son reales. Solo se simula el primer error HTTP 500 del tercer recorrido. El éxito, el reintento y la consulta de persistencia no usan respuestas ficticias.

GitHub Actions

Resultado exitoso verificado mediante API de GitHub
https://github.com/jefroMMM/E2E-Tarea/actions/runs/40037388136

El workflow se activa con push y pull request, instala dependencias, prepara datos, espera la API y ejecuta Cypress headless y el orden inverso. Guarda capturas y videos como artifacts incluso cuando hay fallos. El archivo github-actions.json conserva el resultado consultado y los pasos del job.

Evidencia visual de GitHub Actions



Captura de la ejecución pública verificada. El enlace directo está en la primera página.